14

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

# 컨트롤러와 라우팅

제2부 NestJS 핵심 · 4장 구성 요소

요청 주소 하나가 메서드 하나에 닿기까지



© 한상훈 NestJS 백엔드 마스터

# 200인데 비어 있는 응답

터미널

```
$ curl -i localhost:4014/performances/upcoming
HTTP/1.1 200 OK
X-Powered-By: Express
Connection: keep-alive
Keep-Alive: timeout=65
Transfer-Encoding: chunked
```

→ 회차 두 개 등록 뒤 '다가오는 회차' 조회

→ 에러 로그 없음, 상태 코드 200, 본문 0바이트

© 한상훈 NestJS 백엔드 마스터

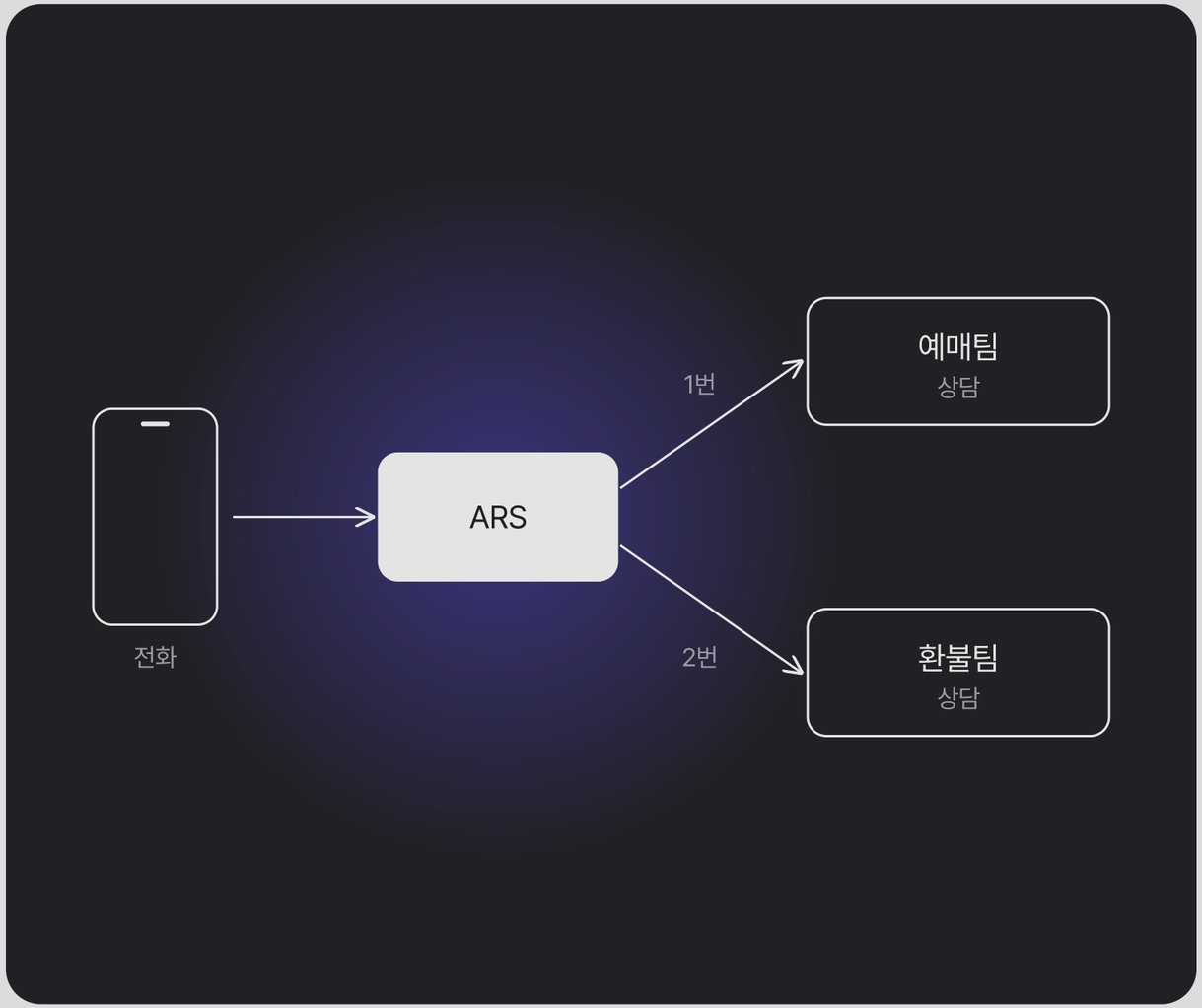
# 콜센터 ARS

→ "예매는 1번, 환불은 2번"

→ ARS: 번호를 듣고 담당 부서로 연결

→ 상담은 담당 부서

→ 번호표가 잘못되면: 환불 문의가 예매 부서로



© 한상훈 NestJS 백엔드 마스터

# 컨트롤러

→ 요청을 받아 서비스로 넘기는 클래스

→ 서비스의 결과를 HTTP 응답으로 변환

→ 업무 판단: 서비스 몫

→ 라우팅: 요청의 메서드와 경로로 처리할 메서드를 고르는 일

→ 예: `/performances` + GET → 목록, `/performances` + POST → 등록

→ 컨트롤러 인스턴스: 서버에 하나, 요청마다 메서드 호출

© 한상훈 NestJS 백엔드 마스터

# 접두어와 경로

performances.controller.ts

```ts
@Controller('performances')
export class PerformancesController {
  constructor(private readonly performancesService: PerformancesService) {}

  @Post()
  create(@Body() createPerformanceDto: CreatePerformanceDto) {
    return this.performancesService.create(createPerformanceDto);
  }

  @Get(':id')
  findOne(@Param('id') id: string) {
    return this.performancesService.findOne(+id);
  }
}
```

→ `@Controller('performances')`: 이 클래스의 모든 경로 앞에 `/performances`

→ `@Get(':id')`: `/performances/:id`, `:id`는 경로 파라미터

→ `:id`: 슬래시 사이 한 칸만 (`/performances/7/ics`는 불일치)

→ 접두어의 파라미터: 예 `@Controller('performances/:performanceId/seats')`

© 한상훈 NestJS 백엔드 마스터

# 메서드 데코레이터

| 데코레이터 | HTTP 메서드 | 회차 API의 예 |
| --- | --- | --- |
| `@Get` | GET | 목록, 하나 조회 |
| `@Post` | POST | 회차 등록 |
| `@Patch` / `@Put` | PATCH / PUT | 회차 수정 |
| `@Delete` | DELETE | 회차 삭제 |

© 한상훈 NestJS 백엔드 마스터

# 부팅 로그의 라우트 표

부팅 로그

```
LOG [RoutesResolver] PerformancesController {/performances}:
LOG [RouterExplorer] Mapped {/performances, POST} route
LOG [RouterExplorer] Mapped {/performances, GET} route
LOG [RouterExplorer] Mapped {/performances/:id, GET} route
LOG [RouterExplorer] Mapped {/performances/:id, PATCH} route
LOG [RouterExplorer] Mapped {/performances/:id, DELETE} route
```

→ 컨트롤러 하나에 한 줄, 라우트 하나에 한 줄

→ 요청이 404일 때 먼저 볼 곳

© 한상훈 NestJS 백엔드 마스터

# 반환값이 응답이 되는 방식

터미널

```
$ curl -i localhost:4014/
Content-Type: text/html; charset=utf-8
예매 API: A구역 3열 15번

$ curl -i localhost:4014/performances
Content-Type: application/json; charset=utf-8
[]

$ curl -s -o /dev/null -w '%{http_code}' -X POST localhost:4014/performances -d '...'
201
```

→ 문자열: 그대로, `text/html`

→ 객체·배열: JSON으로 바꿔서

→ 기본 상태 코드: POST 201, 나머지 200

© 한상훈 NestJS 백엔드 마스터

# upcoming을 아래에 둔 코드

performances.controller.ts

```
  @Get(':id')
  findOne(@Param('id') id: string) {
    return this.performancesService.findOne(+id);
  }

  @Get('upcoming')
  findUpcoming() {
    return this.performancesService.findUpcoming();
  }
```

→ findUpcoming: 시작 시각이 지금보다 뒤인 회차 목록

→ 이 순서 그대로 실행

© 한상훈 NestJS 백엔드 마스터

# 로그에 찍힌 순서

부팅 로그

```
Mapped {/performances, POST} route
Mapped {/performances, GET} route
Mapped {/performances/:id, GET} route
Mapped {/performances/upcoming, GET} route
Mapped {/performances/:id, PATCH} route
Mapped {/performances/:id, DELETE} route
```

→ /performances/:id가 /performances/upcoming보다 위

© 한상훈 NestJS 백엔드 마스터

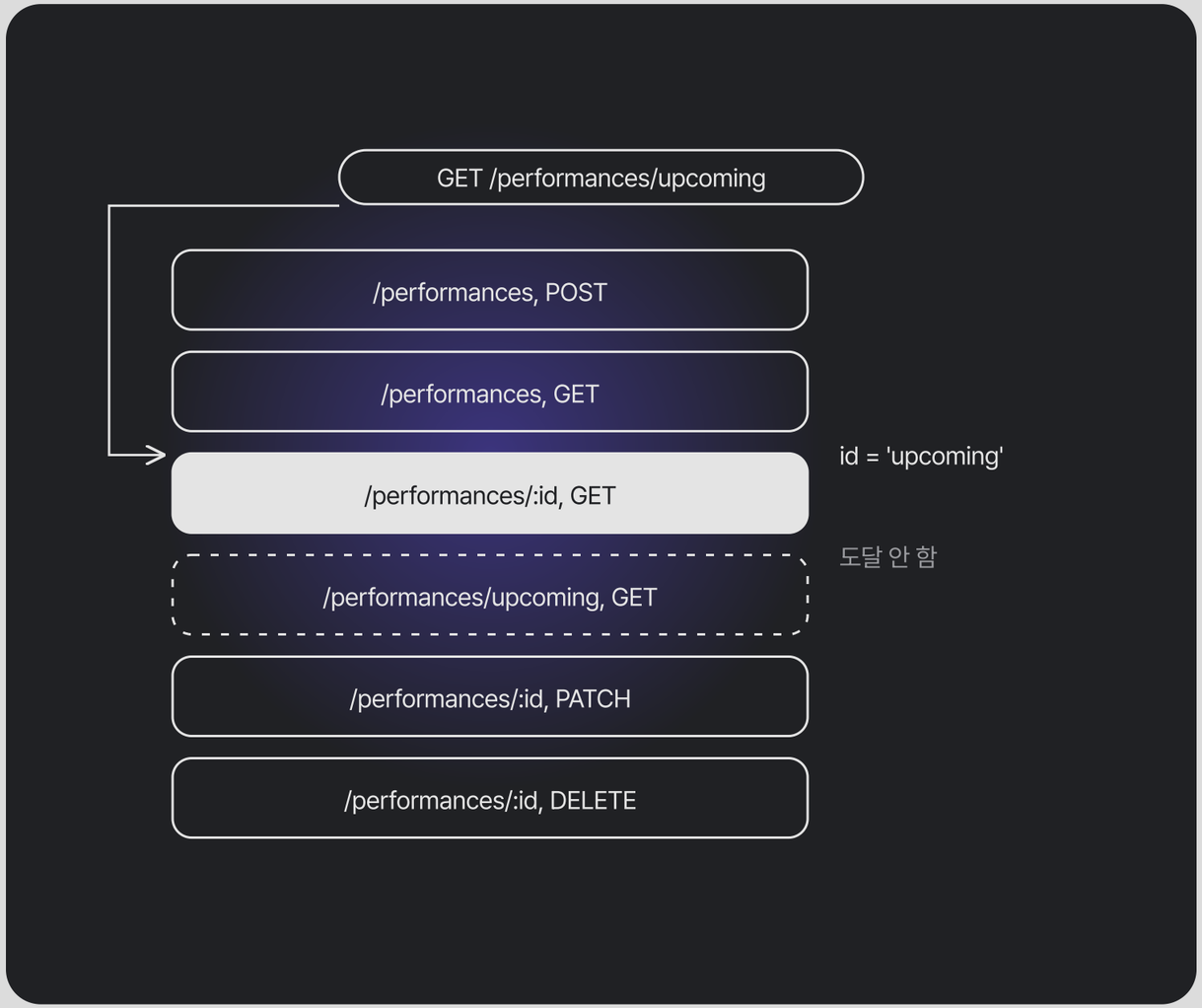
# 위에서부터 맞춰 보기

→ 요청: `GET /performances/upcoming`

→ 첫 번째로 맞는 라우트: `/performances/:id` (id = 'upcoming')

→ `+'upcoming'` = NaN → `findOne(NaN)` → undefined

→ undefined 반환: 200, 본문 없음



© 한상훈 NestJS 백엔드 마스터

# 고정 경로를 위로

```
  @Get('upcoming')
  findUpcoming() { ... }

  @Get(':id')
  findOne(...) { ... }
```

○ ○ ○ 부팅 로그와 요청

```
Mapped {/performances/upcoming, GET} route
Mapped {/performances/:id, GET} route

$ curl localhost:4014/performances/upcoming
[{"id":1,"concertId":1,"startsAt":"2026-11-14T19:00"},{"id":2,"concertId":1,"startsAt":"2026-11-15T18:00"}]
```

→ 같은 컨트롤러 안: 고정 경로가 파라미터 경로보다 먼저

© 한상훈 NestJS 백엔드 마스터

# 값을 꺼내는 데코레이터

| 데코레이터 | 꺼내는 곳 | 예 |
| --- | --- | --- |
| `@Param('id')` | 경로 | `/performances/7` → '7' |
| `@Query('concertId')` | 쿼리 문자열 | `?concertId=1` → '1' |
| `@Body()` | 요청 본문 | JSON을 파싱한 객체 |

→ 경로와 쿼리: 항상 문자열

© 한상훈 NestJS 백엔드 마스터

# 비어 버린 회차 목록

```
  @Get()
  findAll(@Query('concertId') concertId: number) {
    return this.performancesService.findAll(concertId);
  }
  // 서비스: this.performances.filter((p) => p.concertId === concertId)
```

○ ○ ○ 터미널

```
$ curl localhost:4014/performances
[{"id":1,"concertId":1,...},{"id":2,"concertId":1,...}]
$ curl localhost:4014/performances?concertId=1
[]
```

→ 타입 표시 number, 실제 값 '1'

→ 1 === '1': false

© 한상훈 NestJS 백엔드 마스터

# +id와 NaN

```
findOne(@Param('id') id: string) {
  return this.performancesService.findOne(+id);
}
```

→ 스키매틱이 만든 숫자 변환: `+id`

→ `+'7'` = 7, `+'abc'` = NaN

→ NaN으로 찾기: 결과 없음, 에러 없음

© 한상훈 NestJS 백엔드 마스터

# ParseIntPipe

```
  @Get()
  findAll(@Query('concertId', new ParseIntPipe({ optional: true })) concertId?: number) {...}

  @Get(':id')
  findOne(@Param('id', ParseIntPipe) id: number) {...}
```

○ ○ ○ 터미널

```
$ curl localhost:4014/performances?concertId=1
[{"id":1,"concertId":1,...},{"id":2,"concertId":1,...}]
$ curl localhost:4014/performances/abc
{"type":"about:blank","title":"Bad Request","status":400,
 "detail":"Validation failed (numeric string is expected)","instance":"/performances/abc"}
```

→ `ParseIntPipe`: Nest 내장, 문자열 → 정수, 실패 시 400

→ `optional`: 값이 없으면 통과

→ abc: 모양이 틀린 요청 → 400 / 99: 없는 회차 → 404

© 한상훈 NestJS 백엔드 마스터

# 본문의 문자열 숫자

터미널

```
$ curl -X POST localhost:4014/performances -d '{"concertId":"1","startsAt":"2026-11-14T19:00"}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"공연 1번이 없습니다","instance":"/performances"}
```

→ 본문 JSON: 숫자는 숫자, 문자열은 문자열 그대로

→ "1"(문자열)과 1(숫자) 비교 → 공연을 못 찾음

→ 본문 값 검사: 별도 단계 필요

© 한상훈 NestJS 백엔드 마스터

# 없는 회차의 200

터미널

```
$ curl -i localhost:4014/performances/99
HTTP/1.1 200 OK
X-Powered-By: Express
Transfer-Encoding: chunked
```

→ `findOne(99)` → undefined → 200, 본문 없음

→ 앱 쪽: JSON 파싱 실패 → 화면 오류

→ 서버 로그: 아무것도 없음

© 한상훈 NestJS 백엔드 마스터

# NotFoundException

```
  findOne(id: number) {
    const performance = this.performances.find((p) => p.id === id);
    if (!performance) {
      throw new NotFoundException(`회차 ${id}번이 없습니다`);
    }
    return performance;
  }
```

○ ○ ○ 터미널

```
$ curl localhost:4014/performances/99
{"type":"about:blank","title":"Not Found","status":404,"detail":"회차 99번이 없습니다","instance":"/performances/99"}
```

→ 서비스: "없다"를 예외로 / Nest: 404 응답으로

→ 에러 본문: 전역 Problem Details 필터의 모양, detail(예외에 넣은 문장), title(상태 이름), status(숫자)

© 한상훈 NestJS 백엔드 마스터

# 상태 코드를 정하는 곳

| 상태 | 이 강의 회차 API | 정하는 방법 |
| --- | --- | --- |
| 200, 201 | 조회, 등록 | 기본값 |
| 204 | 삭제 | `@HttpCode(204)` |
| 400 | 숫자가 아닌 id | `ParseIntPipe` |
| 404 | 없는 회차 | `NotFoundException` |

→ 상태 코드: 응답을 받은 프로그램이 다음에 할 일을 정하는 숫자

© 한상훈 NestJS 백엔드 마스터

# DELETE와 204

```
  @Delete(':id')
  @HttpCode(204)
  remove(@Param('id', ParseIntPipe) id: number) {
    this.performancesService.remove(id);
  }
```

○ ○ ○ 터미널

```
$ curl -i -X DELETE localhost:4014/performances/2
HTTP/1.1 204 No Content
```

→ 204: 성공, 돌려줄 본문 없음

→ 없는 회차 삭제: remove 안의 `findOne` → 404

© 한상훈 NestJS 백엔드 마스터

# 응답 객체를 직접 받으면

```
@Get(':id/ics')
downloadIcs(@Param('id', ParseIntPipe) id: number, @Res() res: Response) {
  res.setHeader('Content-Type', 'text/calendar');
  return this.performancesService.toIcs(id);
}
```

→ 목적: 회차를 캘린더 파일(.ics)로 내려받기, Content-Type 지정

→ `@Res()`: Express의 응답 객체

→ 이 메서드의 return 값: 응답으로 안 나감

© 한상훈 NestJS 백엔드 마스터

# 3초 뒤 끊긴 요청

터미널

```
$ curl -sS -m 3 localhost:4014/performances/1/ics
curl: (28) Operation timed out after 3006 milliseconds with 0 bytes received
```

→ `-m 3`: curl이 3초 뒤 포기

→ 서버 쪽: 응답을 보낼 코드 없음, 연결 유지

→ 실제 서비스: 앱이나 프록시가 포기할 때까지 대기

© 한상훈 NestJS 백엔드 마스터

# @Header로

```
@Get(':id/ics')
@Header('Content-Type', 'text/calendar')
downloadIcs(@Param('id', ParseIntPipe) id: number) {
  return this.performancesService.toIcs(id);
}
```

○ ○ ○ 터미널

```
$ curl -i localhost:4014/performances/1/ics
HTTP/1.1 200 OK
Content-Type: text/calendar; charset=utf-8

BEGIN:VEVENT
DTSTART:20261114T190000
END:VEVENT
```

© 한상훈 NestJS 백엔드 마스터

# passthrough

```
  downloadIcs(
    @Param('id', ParseIntPipe) id: number,
    @Res({ passthrough: true }) res: Response,
  ) {
    res.setHeader('Content-Type', 'text/calendar');
    return this.performancesService.toIcs(id);
  }
```

→ `passthrough`: 응답 객체는 쓰되, return 값은 Nest가 응답으로

→ 실행 결과: `@Header`와 같은 200, `text/calendar`

→ 응답 객체가 꼭 필요할 때만 (예: 값에 따라 헤더가 바뀔 때)

→ 옵션 없는 `@Res()`: Nest의 응답 처리(JSON 변환, 응답을 가로채는 기능) 전부 포기

© 한상훈 NestJS 백엔드 마스터

# 지금의 회차 라우트

부팅 로그

```
Mapped {/performances, POST} route
Mapped {/performances, GET} route
Mapped {/performances/upcoming, GET} route
Mapped {/performances/:id, GET} route
Mapped {/performances/:id/ics, GET} route
Mapped {/performances/:id, PATCH} route
Mapped {/performances/:id, DELETE} route
```

© 한상훈 NestJS 백엔드 마스터

# 컨트롤러가 하는 일

→ 경로 맞추기: 고정 경로를 파라미터 경로보다 위에

→ 값 꺼내 바꾸기: 경로·쿼리는 문자열, 숫자로 변환

→ 서비스 부르기: 업무 판단은 서비스

→ 응답 정하기: 상태 코드, 헤더, 본문

→ 하지 않는 일: 업무 판단 (예: 팔린 좌석인지)

→ 확인: `curl -i`로 상태 코드와 헤더까지

© 한상훈 NestJS 백엔드 마스터

# 새 라우트를 붙일 때

→ 예: 회차의 남은 좌석 수 API

→ 경로: `GET /performances/:id/remaining` (칸 셋, `:id` 라우트와 불일치)

→ id: `ParseIntPipe`

→ 없는 회차: 서비스의 `NotFoundException` → 404

→ 응답: `{ "remaining": 숫자 }` 객체 return → JSON, 200

© 한상훈 NestJS 백엔드 마스터

# 컨트롤러의 규칙

"컨트롤러는 요청을 받아 서비스에 넘기고, 결과를 HTTP 응답으로 바꾸는 일만 합니다."